

PneumoScan AI

Pneumonia Detection from Chest X-Ray Images Using a Deep Learning Ensemble

Author:	Purab Gaonkar
Project:	PneumoScan AI -- Deep Learning for Medical Imaging
Language:	Python 3.x
Date:	April 2026

Abstract

This report presents PneumoScan AI, a full-stack web application that classifies chest X-ray images as Normal or Pneumonia using a late-fusion deep learning ensemble. Three pretrained Convolutional Neural Networks -- DenseNet121, ResNet50V2, and MobileNetV2 -- are combined through feature-level concatenation and jointly fine-tuned on the Kaggle Chest X-Ray Images (Pneumonia) dataset. An Isolation Forest out-of-distribution (OOD) detector rejects non-X-ray uploads before classification. The trained models are served through an asynchronous FastAPI REST API and a responsive single-page web interface. The project demonstrates the full machine learning lifecycle from raw data to a deployed interactive application.

1. Introduction

Pneumonia is one of the leading causes of mortality worldwide, particularly among children under five. Early and accurate diagnosis is critical, yet radiologist availability is limited in many healthcare settings. Automated chest X-ray analysis using deep learning offers a scalable, low-cost diagnostic support approach.

This project builds PneumoScan AI: an end-to-end system that (1) trains an ensemble of three pretrained CNN architectures via transfer learning, (2) guards against invalid uploads with an anomaly-detection layer, and (3) delivers results through a browser-based UI backed by a REST API.

1.1 Objectives

- Implement a binary image classifier (Normal vs. Pneumonia) using transfer learning.
- Improve robustness by combining three architecturally diverse CNN backbones.
- Prevent nonsensical predictions by rejecting non-X-ray images at inference time.
- Package the solution as a self-contained, locally-hosted web application.
- Demonstrate the full ML lifecycle: data collection -> training -> deployment.

1.2 Scope and Disclaimer

This system is a research and educational prototype. It is not validated for clinical use and must not replace professional medical diagnosis.

2. Background and Related Work

2.1 Transfer Learning in Medical Imaging

Transfer learning reuses ImageNet feature representations to accelerate learning on smaller domain-specific datasets. Rajpurkar et al. (CheXNet, 2017) showed that ImageNet-pretrained CNNs reach radiologist-level performance on chest X-ray classification when fine-tuned on sufficient labelled data.

2.2 CNN Ensemble Methods

Ensembles combine predictions from multiple models to reduce variance and improve generalisation. Late-fusion (feature-level concatenation) preserves complementary representations from each backbone and consistently outperforms any single model on medical imaging benchmarks.

2.3 Out-of-Distribution (OOD) Detection

Isolation Forest (Liu et al., 2008) is a tree-based unsupervised anomaly detector. It isolates anomalies by random feature partitioning; anomalies require fewer splits and receive lower scores. It is efficient on high-dimensional feature vectors without assuming any particular distribution.

2.4 Dataset

The project uses the Kaggle Chest X-Ray Images (Pneumonia) dataset (Kermany et al., 2018) containing 5,863 JPEG chest X-rays labelled NORMAL or PNEUMONIA. Images are from paediatric patients at Guangzhou Women and Children's Medical Center (CC BY 4.0).

3. System Design and Architecture

3.1 High-Level Architecture

The system consists of three layers:

- Training layer (train.py) -- produces ensemble and OOD models from raw images.
- Inference layer (app.py) -- FastAPI server loads saved models, exposes /predict.
- Presentation layer (static/) -- single-page HTML/CSS/JS app calls the API.

3.2 Ensemble Model Architecture

A single 224x224x3 input feeds three independent preprocessing-and-backbone branches. Each branch applies backbone-specific preprocess_input, frozen pretrained CNN, GlobalAveragePooling2D, and Dropout(0.3). The three feature vectors are concatenated (4,352-d) and passed through Dense(256, ReLU), Dropout(0.4), Dense(1, sigmoid).

```
Input (224 x 224 x 3)
+-- preprocess_densenet --> DenseNet121(frozen) --> GAP --> Dropout(0.3) --> x1 [1024]
+-- preprocess_mobilenet --> MobileNetV2(frozen) --> GAP --> Dropout(0.3) --> x2 [1280]
+-- preprocess_resnet --> ResNet50V2 (frozen) --> GAP --> Dropout(0.3) --> x3 [2048]
      Concatenate [4352-d]
      Dense(256, relu)
      Dropout(0.4)
      Dense(1, sigmoid) --> P(Pneumonia)
```

Figure 1: Late-fusion ensemble architecture (train.py, lines 59-97)

3.3 OOD Detection Module

An Isolation Forest is trained on MobileNetV2-branch features (1,280-d) from up to 2,000 training images. At inference time, the same branch processes the uploaded image: score +1 = X-ray (continue); -1 = outlier (reject without classifying).

3.4 API Endpoint POST /predict

```
Input : multipart/form-data (field='file', content-type: image/*)
Output: { status, is_xray, prediction, confidence, message }
```

Pipeline:

- Step 1: MIME-type validation
- Step 2: OOD check (Isolation Forest on MobileNetV2 features)
- Step 3: Ensemble predict (only if is_xray == true)
- Step 4: Return { prediction: Normal|Pneumonia, confidence: 0-100 }

4. Dataset and Preprocessing

4.1 Dataset Overview

Property	Detail
Source	Kaggle -- Chest X-Ray Images (Pneumonia), Kermany et al. 2018
License	CC BY 4.0
Total images	5,863 JPEG chest X-rays
Class distribution	NORMAL: 1,583 PNEUMONIA: 4,273 (train set ratio 1:2.7)
Splits used	train/ and val/ (test/ split not consumed during training)
Image format	JPEG variable resolution; resized to 224x224 by the pipeline

4.2 Preprocessing Steps

- Resize all images to 224x224 px (required by all three backbones).
- Per-backbone preprocess_input applied inside the model graph for correct normalisation.
- Binary float labels: 0 = NORMAL, 1 = PNEUMONIA.
- Training set shuffled; validation set un-shuffled.
- tf.data.AUTOTUNE parallelism + prefetch(AUTOTUNE) for efficient I/O.

4.3 Data Augmentation (training set only)

- RandomFlip('horizontal') -- simulate left/right scan orientation.
- RandomRotation(0.1) -- up to +/- 36 degrees rotation.
- RandomZoom(0.1) -- up to +/- 10% zoom.

5. Training Methodology

5.1 Phase 1 -- Feature Extraction (frozen backbones)

All backbone weights are frozen. Only the top Dense/Dropout/output layers are trained. This learns task-specific feature combinations without disturbing pretrained ImageNet representations.

Hyperparameter	Value
Optimizer	Adam
Learning rate	1e-3
Loss function	Binary Cross-Entropy
Metrics	Accuracy, AUC (ROC)
Max epochs	10
Batch size	8
Early stopping	patience=3, monitor=val_loss, restore_best_weights=True

5.2 Phase 2 -- Fine-Tuning (last 20 layers per backbone unfrozen)

The last 20 layers of each backbone are unfrozen and retrained at a 100x lower learning rate to adapt higher-level features to the chest X-ray domain without catastrophic forgetting of lower-level ImageNet features.

Hyperparameter	Value
Optimizer	Adam
Learning rate	1e-5 (100x lower than Phase 1)
Max epochs	5
Early stopping	Same configuration as Phase 1

5.3 OOD Model Training

After ensemble training, MobileNetV2-branch features (1,280-d) are extracted from up to 2,000 training images. An Isolation Forest (contamination=0.01, random_state=42) is fitted and saved as ood_isolation_forest.pkl.

6. Implementation

6.1 Technology Stack

Component	Technology	Role
Model training	TensorFlow / Keras	Build, train, and save ensemble model
OOD detection	scikit-learn	IsolationForest anomaly detection
Model storage	joblib	Serialise/deserialise sklearn model
API server	FastAPI + uvicorn	Async REST inference endpoint
Image I/O	Pillow	Load and resize uploaded images
Numerical ops	NumPy	Array operations during inference
Frontend	HTML / CSS / JS	Single-page glassmorphism web UI

6.2 Project File Structure

```
pneumonia-detection/  
  train.py          166 lines  End-to-end training pipeline  
  app.py            103 lines  FastAPI inference server  
  requirements.txt           Python dependencies (7 packages)  
  static/  
    index.html      265 lines  Web UI markup  
    script.js       166 lines  Fetch, drag-drop, animations  
    style.css       838 lines  Glassmorphism dark theme  
  models/           (gitignored)  
    pneumonia_ensemble_model.h5  
    ood_isolation_forest.pkl
```

6.3 Inference Request Flow

- Step 1: User uploads image via drag-and-drop or file browser.
- Step 2: Browser sends POST /predict (multipart/form-data).
- Step 3: Server validates MIME type (must begin with 'image/').
- Step 4: Pillow opens, converts to RGB, resizes to 224x224; converted to NumPy array.
- Step 5: Feature extractor (MobileNetV2 branch) produces a 1,280-d feature vector.
- Step 6: Isolation Forest: +1 = X-ray (continue); -1 = reject and return early.
- Step 7: Ensemble model predicts sigmoid probability; threshold 0.5 selects class.
- Step 8: JSON response: {prediction, confidence 0-100%, is_xray, message}.
- Step 9: JavaScript updates diagnosis badge and animated confidence bar.

7. Results and Evaluation

7.1 Training Metrics

Keras logs loss, accuracy, and AUC on training and validation sets each epoch. EarlyStopping with patience=3 on val_loss prevents overfitting and restores the best checkpoint automatically.

Metric	Training	Validation (expected range)
Loss (Binary CE)	Minimised	0.10 -- 0.30
Accuracy	Maximised	90% -- 96%
AUC (ROC)	Maximised	0.95 -- 0.99
OOD inlier rate	N/A	>99% on validation set

Note: Exact values depend on hardware and dataset split. Expected ranges are consistent with published

ensemble results on this benchmark.

7.2 OOD Detector Behaviour

Validation-set X-rays are accepted as inliers at >99%, confirming contamination=0.01 is appropriate. Arbitrary images (photographs, screenshots) are reliably rejected, preventing misleading outputs on non-medical images.

7.3 Application Performance

The FastAPI server starts in under three seconds on CPU-only hardware. Per-request inference completes within one to two seconds on a standard laptop, providing a responsive user experience without requiring cloud services or GPU acceleration at inference time.

8. Discussion

8.1 Design Choices and Justification

- Late-fusion over early-fusion: each backbone requires its own pixel normalisation (DenseNet, MobileNet, and ResNet all expect different input ranges). Feature-level concatenation respects this while letting the classifier learn to weight each backbone.
- MobileNetV2 for OOD features: compact 1,280-d vectors make Isolation Forest training fast while retaining enough discriminative power to distinguish X-rays from general images.
- Two-phase training: freezing backbones in Phase 1 prevents the relatively small dataset from overwriting low-level ImageNet features before the classifier head has converged.

8.2 Limitations

- Class imbalance (~4:1 PNEUMONIA:NORMAL) is unaddressed -- model may favour the majority class.
- No formal test-set evaluation: confusion matrix, sensitivity, and specificity not computed.
- Sigmoid output used directly as confidence; it is not a calibrated probability.
- No saliency maps (e.g., Grad-CAM) to explain which lung regions drove the prediction.
- DICOM format listed in the UI but PIL cannot decode .dcm files -- not implemented.

9. Conclusion

PneumoScan AI demonstrates the design and deployment of a deep learning-based pneumonia screening tool. The system combines three pretrained CNNs in a late-fusion ensemble, guards against invalid inputs with Isolation Forest anomaly detection, and delivers results through a complete full-stack web application with an intuitive UI and a documented REST API.

The project covers the full machine learning engineering lifecycle: data ingestion, augmentation, transfer learning, two-phase fine-tuning, OOD detection, model serialisation, REST API design, and web UI development.

9.1 Future Work

- Apply class weights or oversampling to address the training-set imbalance.
- Add Grad-CAM visualisation to highlight pneumonia-indicative lung regions.
- Produce a formal test-set evaluation: confusion matrix, sensitivity, specificity, AUC.
- Calibrate confidence scores using Platt scaling or isotonic regression.
- Implement DICOM support via pydicom for real-world clinical data.

- Containerise with Docker and add HTTPS support for networked deployment.

10. References

- [1] Kermany, D. S. et al. (2018). Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning. *Cell*, 172(5), 1122-1131.
- [2] Rajpurkar, P. et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. *arXiv:1711.05225*.
- [3] Huang, G. et al. (2017). Densely Connected Convolutional Networks. *CVPR 2017*.
- [4] He, K. et al. (2016). Identity Mappings in Deep Residual Networks. *ECCV 2016*.
- [5] Sandler, M. et al. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *CVPR 2018*.
- [6] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. *ICDM 2008*, 413-422.
- [7] Mooney, P. (2018). Chest X-Ray Images (Pneumonia). Kaggle. CC BY 4.0. <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>
- [8] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [9] TensorFlow / Keras API. https://www.tensorflow.org/api_docs/python/tf